Food Waste Intelligence Platform

منصة ذكية لتحليل والتنبؤ بفقد وهدر الغذاء



Food Waste Intelligence Platform

منصة ذكية لتحليل والتنبؤ بفقد وهدر الغذاء

---

1. الفكرة الرئيسية (Main Idea)

فكرة المشروع هي بناء منصة ذكية بتستخدم **Data Engineering** و**Data Science** و**Machine Learning** علشان تحلل فقد الغذاء، وتكتشف الأنماط والعوامل المرتبطة بيه، وتساعدنا نتوقع نسب الفقد المحتملة ونحدد المنتجات أو مراحل سلسلة الإمداد الأكثر عرضة للهدر.

المنصة هتبدأ باستخدام بيانات فقد الغذاء المتاحة عن مصر، وبعد كده هنحوّل البيانات إلى نظام بيحاكي وصول الأحداث في الوقت الفعلي (Simulated Real-Time Data Pipeline).

هنستخدم البيانات التاريخية المتاحة كأساس للتحليل، ونقدر نولّد بيانات صناعية إضافية عند الحاجة علشان نختبر الـ Pipeline على حجم أكبر من البيانات. وبعد كده نطوّر النظام بحيث يستقبل بيانات جديدة باستمرار من مصادر فعلية، لما تكون متاحة.

الهدف النهائي هو إنشاء نظام متكامل بيجمع بين تحليل البيانات، ومعالجتها، وتخزينها، والتنبؤ بالمخاطر، وعرض النتائج في Dashboard تفاعلية.

2. ليه اخترنا الفكرة دي؟ (Why This Idea?)

فقد الغذاء مشكلة بتأثر على الاقتصاد، واستخدام الموارد، وكفاءة سلاسل الإمداد الغذائية.

خلال مراحل إنتاج الغذاء وتخزينه ونقله وتوزيعه، ممكن يحصل فقد بسبب عوامل مختلفة، زي ظروف التخزين، وطريقة الحصاد، والتعامل مع المنتجات، ومشكلات النقل أو التوزيع.

لكن وجود بيانات عن الفقد لوحده مش كفاية؛ إحنا محتاجين نحوّل البيانات دي لمعلومات تساعدنا نفهم المشكلة ونتعامل معاها بشكل أفضل.

اخترنا المشروع للأسباب دي:

- **أهمية المشكلة:** تقليل فقد الغذاء بيساعد على تحسين استخدام الموارد وتقليل الخسائر الاقتصادية.
- **الاستفادة من البيانات:** نقدر نحلل نسب الفقد حسب المحصول والسنة ومرحلة سلسلة الإمداد والمصدر المتاح.
- **تطبيق Data Engineering عمليًا:** نقدر نبني Pipeline لجمع البيانات وتنظيفها ومعالجتها وتخزينها.
- **استخدام Machine Learning:** نقدر ندرس إمكانية التنبؤ بمستويات الفقد، بشرط توافر بيانات كافية وموثوقة.
- **دعم اتخاذ القرار:** نقدر نعرض الأنماط والمخاطر في Dashboard تساعد المستخدم يفهم البيانات ويحدد أولويات التحسين.
- **قابلية التطوير:** نقدر نبدأ ببيانات تاريخية، ثم نضيف محاكاة للبيانات، وبعدها نربط النظام بمصادر بيانات فعلية لو متاحة.

3. إيه المشكلة اللي بنحلها؟ (The Problem)

المشكلة مش بس إن فيه فقد غذاء، لكن كمان إن البيانات المتاحة ممكن تكون تاريخية، أو غير مكتملة، أو مختلفة في طرق القياس والتسجيل.

وده بيخلق تحديات زي:

1. صعوبة فهم اختلاف نسب الفقد بين المحاصيل والسنوات ومراحل سلسلة الإمداد.
2. وجود قيم مفقودة في بعض الأعمدة، زي كمية الفقد بالطن وكمية الفقد لكل هكتار.
3. اختلاف طرق جمع البيانات والمراجع المستخدمة، وده ممكن يأثر على إمكانية مقارنة النتائج.
4. صعوبة اختبار نظام معالجة بيانات لحظي باستخدام ملف CSV ثابت فقط.
5. الحاجة إلى معرفة إذا كان ممكن استخدام البيانات المتاحة لبناء نموذج تنبؤ موثوق.
6. صعوبة تحويل نتائج التحليل إلى معلومات واضحة تساعد في تحديد أولويات التدخل.

المشكلة التقنية

البيانات التاريخية بتدينا صورة عن اللي حصل في الماضي، لكنها مش بتوصل تلقائيًا مع حدوث أحداث جديدة.

علشان كده، لو عايزين نبني نظام بيستقبل البيانات ويعالجها باستمرار، محتاجين نصمم Data Pipeline تدعم تدفق البيانات، حتى لو هنبدأ بمحاكاة الأحداث باستخدام البيانات المتاحة.

ومن المهم نفرّق بين محاكاة تدفق البيانات وبين الحصول على بيانات حقيقية لحظية؛ لأن المحاكاة بتختبر النظام، لكنها مش دليل على إن البيانات الجديدة بتمثل أحداثًا حقيقية.

4. هنحل المشكلة إزاي؟ (Proposed Solution)

هنبني منصة متكاملة على مراحل.

المرحلة الأولى: فهم البيانات وتجهيزها

هنبدأ ببيانات فقد الغذاء المتاحة عن مصر، واللي بتحتوي على معلومات زي:

- الدولة والمحصول وكود التصنيف.
- السنة ونسبة الفقد.
- مرحلة سلسلة الإمداد.
- طريقة جمع البيانات والمرجع.
- كمية الفقد بالطن، لما تكون متاحة.
- المستوى الجغرافي، لما يكون موجودًا في البيانات.

هنراجع جودة البيانات، ونتأكد من أنواع الأعمدة، والقيم المفقودة، والتكرارات، والقيم غير المعتادة.

مش هنحذف أي سجل أو نعتبره خطأ لمجرد إنه مختلف عن باقي السجلات؛ هنراجع السبب قبل اتخاذ القرار.

المرحلة الثانية: تنظيف البيانات وتجهيزها للتحليل

هنجهز نسخة منظمة من البيانات، مع الحفاظ على البيانات الأصلية للرجوع ليها.

هنعمل الآتي:

- مراجعة القيم المفقودة والتعامل معاها حسب طبيعة كل عمود.
- التحقق من صحة نسب الفقد ونطاق القيم.
- مراجعة التكرارات باستخدام مفتاح مناسب للسجل، بدل ما نحذف سجلات مختلفة لمجرد تشابه بعض الأعمدة.
- توحيد التصنيفات النصية عند الحاجة، من غير تغيير المعنى الأصلي.
- الاحتفاظ بعلامات الجودة، زي القيم المشكوك فيها أو المتطرفة، لمراجعتها بدل حذفها تلقائيًا.
- التمييز بين القيم الأصلية والقيم المشتقة أثناء المعالجة.

المرحلة الثالثة: إنشاء Simulated Real-Time Data

بما إن البيانات الأصلية تاريخية وموجودة في ملف ثابت، هنستخدم Python لإنشاء محاكاة لتدفق البيانات.

الفكرة إن البرنامج يقرأ السجلات بالتدريج، ويضيف لكل سجل وقت وصول محاكى، ويبعت الأحداث إلى نظام المعالجة.

ولو حجم البيانات الحالية مش كافي لاختبار النظام، ممكن نولّد بيانات صناعية إضافية بناءً على قواعد وتوزيعات مدروسة.

لكن لازم نراعي الآتي:

- البيانات الصناعية هتكون مميزة بوضوح عن البيانات الأصلية.
- مش هنعتبر القيم المولدة قياسات حقيقية لفقد الغذاء.
- مش هنكرر السجلات الأصلية ونعتبرها مشاهدات مستقلة جديدة.
- مش هنستخدم البيانات الصناعية لإثبات دقة نموذج التنبؤ على الواقع.
- هنختبر جودة البيانات المولدة ومدى مناسبتها للغرض من المحاكاة.

الهدف من المرحلة دي هو اختبار قدرة النظام على استقبال ومعالجة عدد كبير من الأحداث بشكل مستمر.

المرحلة الرابعة: بناء Data Engineering Pipeline

هنبني مسار لمعالجة البيانات بداية من مصدرها لحد تخزينها واستخدامها.

المسار المقترح:

. **Data Source:** البيانات التاريخية أو البيانات الصناعية الناتجة عن المحاكاة.

. **Data Ingestion:** استقبال الأحداث الجديدة تدريجيًا.

. **Stream Processing:** معالجة الأحداث والتحقق من جودتها أثناء وصولها.

. **Data Storage:** تخزين البيانات الأصلية والمعالجة في قاعدة بيانات أو Data Lake.

. **Analytics:** حساب المؤشرات وتحليل الاتجاهات.

. **Machine Learning:** استخدام البيانات المناسبة لتطوير نموذج تنبؤ عند توافر شروط التدريب والتقييم.

. **Dashboard:** عرض النتائج والتحديثات للمستخدم.

ممكن نستخدم Python في البداية، وبعدها نضيف Apache Kafka لتدفق الأحداث، وSpark Structured Streaming لمعالجتها، وقاعدة بيانات أو Data Lake للتخزين، وأداة زي Power BI لعرض النتائج.

اختيار الأدوات النهائي هيعتمد على حجم البيانات، ومتطلبات المشروع، والموارد المتاحة.

المرحلة الخامسة: تحليل البيانات واستخراج الأنماط

هنحلل البيانات علشان نجاوب عن أسئلة زي:

- إيه المحاصيل اللي عندها أعلى نسب فقد في البيانات المتاحة؟
- إزاي نسب الفقد بتختلف من سنة للتانية؟
- إيه مراحل سلسلة الإمداد اللي بتظهر فيها نسب فقد مرتفعة؟
- هل فيه اختلافات واضحة بين مصادر البيانات وطرق جمعها؟
- إيه المؤشرات اللي نقدر نعتمد عليها في مقارنة المحاصيل أو المراحل؟
- هل البيانات المتاحة كافية لبناء نموذج تنبؤ قابل للتقييم؟

لازم نراعي إن ارتفاع نسبة الفقد في سجل معين مش معناه بالضرورة إن المحصول ده هو الأكثر فقدًا على مستوى مصر كلها؛ لأن المقارنة بتعتمد على نطاق القياس، وجودة البيانات، وتغطية المصادر.

المرحلة السادسة: التنبؤ باستخدام Machine Learning

بعد فهم البيانات وتجهيزها، هندرس إمكانية بناء نموذج يتنبأ بنسبة الفقد أو يصنف مستوى المخاطرة.

النموذج ممكن يستخدم خصائص مناسبة، زي المحصول والسنة ومرحلة سلسلة الإمداد والمعلومات المتاحة عن القياس والمصدر.

لكن قبل بناء النموذج، لازم نتأكد من:

- توافر عدد كافٍ من السجلات المناسبة.
- عدم وجود تسريب للمعلومات من المستقبل إلى بيانات التدريب.
- وجود طريقة صحيحة لتقسيم البيانات إلى تدريب واختبار.
- استخدام مقاييس تقييم مناسبة للمهمة.
- اختبار النموذج على بيانات لم يتدرب عليها.

لو البيانات غير كافية أو مش ممثلة للواقع، هنعتبر التنبؤ جزءًا يحتاج إلى تطوير إضافي، بدل ما ندّعي إن النموذج قادر على توقع الفقد الحقيقي بدقة.

المرحلة السابعة: بناء Dashboard تفاعلية

هنعرض النتائج في Dashboard توضح:

- إجمالي السجلات المستقبلة.
- آخر وقت وصل فيه حدث جديد.

- نسب الفقد حسب المحصول والسنة والمرحلة.
- مؤشرات جودة البيانات والقيم المفقودة.
- نتائج التحليل والتنبؤ، لما تكون متاحة.
- تنبيهات مبنية على قواعد واضحة أو مخرجات النموذج.

وفي حالة استخدام المحاكاة، هنوضح إن التحديثات ناتجة عن بيانات محاكاة، مش عن قياسات لحظية حقيقية.

5. إيه النتيجة المتوقعة؟ (Expected Outcome)

النتيجة المتوقعة هي بناء نموذج أولي لمنصة ذكية قادرة على:

- قراءة بيانات فقد الغذاء وتجهيزها للتحليل.
- اختبار تدفق البيانات باستخدام Simulated Real-Time Events.
- معالجة وتخزين البيانات من خلال Data Engineering Pipeline.
- تحليل أنماط فقد الغذاء واستخراج مؤشرات مفيدة.
- تقييم إمكانية بناء نموذج Machine Learning للتنبؤ.
- عرض النتائج في Dashboard قابلة للتحديث.
- الاستعداد للربط بمصادر بيانات فعلية في مرحلة لاحقة.

المنصة هتكون خطوة نحو دعم اتخاذ القرار المبني على البيانات، لكنها مش هتقدر تثبت تقليل الفقد الفعلي إلا بعد اختبارها في بيئة واقعية وقياس تأثيرها.

7. ملخص الفكرة (Project Summary)

إحنا بنقترح بناء Food Waste Intelligence Platform بتحوّل بيانات فقد الغذاء من مجرد سجلات تاريخية إلى نظام متكامل لمعالجة البيانات وتحليلها ودراسة إمكانية التنبؤ بمستويات الفقد.

هنبدأ ببيانات مصرية متاحة، ونجهزها ونراجع جودتها، وبعدها هنحاكي تدفق البيانات علشان نختبر الـ Data Engineering Pipeline على عدد أكبر من الأحداث. وفي مراحل التطوير التالية، هنضيف أدوات Streaming وMachine Learning وDashboard، مع إمكانية الربط بمصادر فعلية لو متاحة.

الهدف الأساسي هو بناء نظام قابل للتطوير بيساعد على فهم فقد الغذاء بصورة أفضل، وتجهيز البيانات لدعم قرارات مستقبلية مبنية على الأدلة.

الخطة التنفيذية واختيار الأدوات لكل مرحلة

1. مراحل المشروع والـ Features والقرارات الهندسية

ده الجدول الأساسي اللي هنشتغل على أساسه. كل مرحلة فيها الـ Feature اللي هننفذها، والأداة اللي هنختارها فعلًا، والسبب الهندسي للاختيار.

المرحلة	الـ Features اللي هنضيفها	الاختيار الأساسي	ليه اخترنا الأداة دي؟
1. فهم المشكلة	تحديد المستخدمين، أهداف المنصة، ومؤشرات النجاح	GitHub Issues + README	علشان نحوّل الفكرة لمتطلبات واضحة ومهام قابلة للتنفيذ، ونوثّق القرارات من أول يوم.
2. اكتشاف مصادر البيانات	قراءة ملفات مصرية عن فقد الغذاء وتوثيق مصادرها	Python + Pandas	لأن البيانات متاحة مبدئيًا كملفات تاريخية، وPandas مناسبة لقراءة CSV وExcel وفحص الأعمدة والقيم.
3. تقييم جودة البيانات	اكتشاف القيم المفقودة والتكرارات والأنواع غير الصحيحة	Pandas + Pandera	لكتابة Panderaللتحليل والفحص، و Pandas قواعد تحقق قابلة للتكرار بدل الاعتماد على الفحص اليدوي فقط.
4. الاحتفاظ بالبيانات الأصلية	حفظ الملفات الخام وإمكانية إعادة المعالجة	Local file storage في البداية	لأن الملفات الأصلية ثابتة ومحدودة نسبيًا في البداية؛ مش محتاجين ندفع تكلفة تخزين سحابي قبل ما نحتاجه.
5. تنظيف وتحويل البيانات	توحيد التصنيفات، معالجة القيم المفقودة، وإنشاء أعمدة مشتقة	Pandas	لأنها كافية للمعالجة الأولية والبيانات الجدولية المتاحة، وأسهل في التطوير والتحقق من النتائج.
6. تصميم قاعدة البيانات	تخزين السجلات المنظمة والمؤشرات والاستعلام عنها	PostgreSQL	لأن البيانات منظمة، والعلاقات والاستعلامات التجميعية مهمة، وهنحتاج joins وfilters وaggregations.
7. محاكاة وصول الأحداث	إرسال السجلات تدريجيًا مع وقت وصول محاكى	Python Producer	لأن مصدرنا ملف تاريخي، فنقدر نولّد أحداثًا بالتدريج من غير الحاجة إلى نظام Streaming كامل من أول يوم.
8. نقل الأحداث	استقبال الأحداث وتوزيعها على مستهلكين مختلفين	Apache Kafka	لأنه يفصل بين منتج الأحداث ومستهلكيها، ويساعد في الاحتفاظ بالأحداث وإعادة قراءتها وتوسيع النظام.



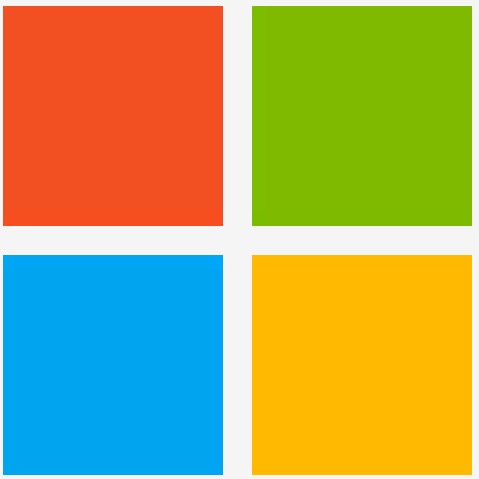
Apache Kafka

9. معالجة الأحداث	التحقق من كل حدث، وتحديد المقبول والمرفوض	Python Consumer في النسخة الأولى	لأن قواعد المعالجة الحالية مش محتاجة محرك موزع. نضيف Spark Structured Streaming لما يكون فيه احتياج فعلي لمعالجة Streaming على نطاق أكبر.
10. التعامل مع الأخطاء	تسجيل الأحداث غير الصالحة وإعادة المحاولة	Python Logging + جدول للأحداث المرفوضة	لأننا محتاجين نعرف الحدث فشل ليه، ونحتفظ به للمراجعة بدل ما نفقده بصمت.
11. التحليل واستخراج المؤشرات	تحليل الفقد حسب المحصول والسنة والمرحلة	SQL + Pandas	مناسب للاستعلامات والتجميع داخل SQL مناسب للتحليل الإضافي وتجهيز النتائج. Pandasقاعدة البيانات، و

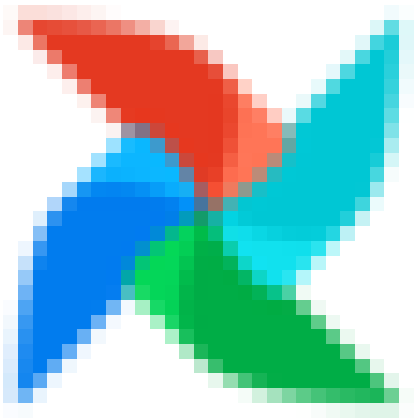


Apache Spark

12. بناء نموذج تنبؤ	تقدير نسبة الفقد أو تصنيف مستوى المخاطر عند توافر بيانات مناسبة	Scikit-learn	لأنه يوفّر أدوات لبناء Baseline ومقارنة النماذج وتقييمها قبل إدخال نماذج أكثر تعقيدًا.
13. عرض النتائج	رسوم تفاعلية وفلاتر ومؤشرات جودة البيانات	Power BI	لأنه مناسب لبناء تقارير تحليلية تفاعلية تربط المؤشرات بالمحاصيل والسنوات والمراحل.
14. جدولة المهام	تشغيل التنظيف والتحديثات والتقييم دوريًا	Apache Airflow عند الحاجة	لأنه مناسب لتنظيم الـ Workflows ومتابعة تنفيذها وإعادة المهام الفاشلة. لكن مش لازم نبدأ به لو عندنا سكريبت بسيط واحد.
15. الاختبارات	اختبار التحويلات وقواعد الجودة وتدفق البيانات	pytest + Pandera	علشان نكتشف أخطاء المعالجة تلقائيًا، ونضمن إن التعديلات الجديدة مش بتكسر النتائج القديمة.



Microsoft Learn



Airflow 3.3.2 Documentation

16. تشغيل المشروع	توحيد بيئة التشغيل بين أعضاء الفريق	Docker Compose	لأنه يقدر يشغّل مكونات المشروع وخدماته بإعدادات متقاربة على أجهزة الفريق.
17. مراقبة النظام	متابعة الأخطاء وعدد الأحداث ووقت آخر تحديث	Python Logging + مؤشرات تشغيلية	نبدأ بالأساسيات، وبعدها نضيف Prometheus وGrafana لو احتجنا مراقبة أوسع.
18. نشر المشروع	إتاحة نسخة قابلة للتجربة والتقييم	GitHub + Docker	لتوثيق الكود، ومراجعة التغييرات، وتجهيز بيئة تشغيل قابلة لإعادة الاستخدام.

مهم: اختيار الأداة مش معناه إننا هنستخدمها من أول يوم. مثلًا Kafka وAirflow وSpark أدوات ضمن التصميم المستهدف، لكن تنفيذها هيكون حسب ترتيب الأولويات واحتياجات كل مرحلة.

2. القرارات الهندسية بالتفصيل: هنختار إيه وليه؟

المرحلة الأولى: فهم المشكلة وتحديد الـ Features

القرار: GitHub Issues لتقسيم العمل، وREADME لتوثيق المشروع.

هنحوّل الفكرة لمجموعة وظائف واضحة:

- استيراد بيانات فقد الغذاء المتاحة عن مصر.
- الاحتفاظ بالنسخة الخام من البيانات.
- فحص جودة البيانات وتسجيل نتائج الفحص.
- تنظيف البيانات وتجهيزها للتحليل.
- محاكاة وصول أحداث جديدة.
- معالجة الأحداث وتخزينها.
- حساب مؤشرات فقد الغذاء.
- تقييم إمكانية التنبؤ بالمخاطر.
- عرض النتائج والتنبيهات في Dashboard.

ليه الاختيار ده؟

لأننا في المرحلة دي بنحل مشكلة تنظيم المشروع، مش مشكلة معالجة ملايين السجلات. مش محتاجين نبدأ بأداة ضخمة لإدارة الـ Workflows قبل ما نحدد أصلًا الـ Workflows اللي هننفذها.

ناتج المرحلة: وثيقة متطلبات، قائمة Features، ومعايير واضحة لنجاح النسخة الأولى.

المرحلة الثانية: اكتشاف البيانات وفحصها

القرار: Python + Pandas.

هنقرأ الملفات المتاحة ونفحص:

- أسماء الأعمدة وأنواعها.
- عدد السجلات.
- عدد القيم المفقودة.
- عدد القيم المختلفة في المحاصيل والسنوات والمراحل.
- نطاقات نسب الفقد.
- وجود سجلات مكررة محتملة.
- المصادر والمراجع وطرق القياس.

مثلًا، لو عندنا الأعمدة التالية:

Crop	Year	Supply Chain Stage	Loss Percentage
Wheat	2022	Storage	8.5
Wheat	2023	Storage	9.2
Tomato	2023	Transport	12.1

نقدر نستخدم Pandas علشان نعرف عدد السجلات لكل محصول، ونحدد السنوات المتاحة، ونكتشف هل في أعمدة ناقصة.

ليه مش Spark من البداية؟

لأننا لسه مش عارفين إن حجم البيانات يستدعي المعالجة الموزعة. تشغيل Spark لمجرد إن المشروع Data Engineering بيضيف إعدادات وتعقيد من غير فائدة مؤكدة.

Feature جديدة: Data Profiling Report يوضح حالة البيانات قبل أي تعديل.

المرحلة الثالثة: جودة البيانات وتنظيفها

القرار: Pandas للمعالجة، وPandera لقواعد التحقق.

هنكتب قواعد زي:

- السنة لازم تكون من النوع الصحيح.

- نسبة الفقد لازم تكون ضمن نطاق مقبول وفق تعريف المصدر.
- كود المحصول لازم يكون متوافقًا مع التصنيف المستخدم.
- السجل لازم يحتفظ بمعلومات المصدر.
- القيم المفقودة تتسجل وتتراجع حسب طبيعة العمود.

مش هنفترض إن أي نسبة أكبر من المعتاد غلط. ممكن تكون حالة حقيقية، أو اختلافًا في طريقة القياس، أو خطأً في المصدر؛ وده محتاج مراجعة.

كمان مش هنحذف سجل لمجرد إن محصوله وسنته زي سجل تاني. هنحتاج نحدد المفتاح المناسب، لأن السجلين ممكن يختلفوا في المصدر أو المنطقة أو مرحلة سلسلة الإمداد.

ليه مش أداة واحدة تعمل كل حاجة؟

تساعدنا في تعريف قواعد الجودة والتحقق منها. الفصل بين المهمتين بيخلّي الاختبارات Pandera مسؤولة عن تحويل البيانات، بينما Pandas أوضح وأسهل في الصيانة.

Features جديدة:

- تقرير جودة البيانات.
- تسجيل أسباب رفض السجلات.
- الاحتفاظ بعلامات الجودة.
- إمكانية إعادة تشغيل التنظيف على البيانات الأصلية.

المرحلة الرابعة: Database ولا Data Lake؟

دي نقطة مهمة جدًا في المشروع، والقرار فيها لازم يعتمد على طبيعة البيانات واستخدامها.

قراري للمشروع: PostgreSQL + Raw File Storage

هنستخدم PostgreSQL لتخزين البيانات المنظمة اللي محتاجين نحللها ونستعلم عنها، ونحتفظ بالملفات الأصلية منفصلة في Raw Storage.

ليه PostgreSQL؟

- البيانات الحالية جدولية ومنظمة.
- هنحتاج نربط السجلات بمصادرها وتصنيفات المحاصيل ومراحل سلسلة الإمداد.
- هنحتاج استعلامات SQL وتجميعات حسب السنة والمحصول والمرحلة.
- محتاجين قيودًا على البيانات ومعاملات قاعدة البيانات.

المرنة Metadata لو احتجنا الاحتفاظ ببعض الـ JSON بيدعم كمان أنواع بيانات PostgreSQL.



PostgreSQL

ليه مش Data Lake فقط؟

لأننا في النسخة الأولى مش محتاجين نبني طبقة تخزين واسعة للملفات الخام والبيانات المتنوعة قبل ما يكون عندنا حجم أو استخدام يستدعي ده. تخزين الملفات الخام في مجلد منظم كفاية كبداية، وقاعدة البيانات هتخدم الاستعلامات والتحليلات المنظمة.

إمتى نضيف Data Lake؟

لو بدأنا نستقبل كميات كبيرة من الملفات التاريخية أو أحداثًا من مصادر متعددة، أو احتجنا نحتفظ بالـ Raw Events بصيغ زي JSON وParquet على نطاق أكبر، ساعتها نضيف Object Storage مثل Amazon S3 أو Azure Data Lake Storage.

فيه فرق لازم نوضحه: الاحتفاظ بالملفات الأصلية مش معناه إننا بنينا Data Lake كامل. ده مجرد قرار عملي لحماية البيانات المصدرية. ممكن نطوّر طبقة التخزين بعدين حسب الحاجة.

Features جديدة:

- تخزين منظم قابل للاستعلام.
- الاحتفاظ بنسخة خام للرجوع ليها.
- إمكانية إعادة بناء البيانات المعالجة.
- الاحتفاظ بمصدر كل سجل وتاريخ معالجته.

المرحلة الخامسة: Simulated Real-Time Pipeline

القرار: Python Producer في البداية، ثم Kafka كطبقة نقل أحداث.

بما إن مصدرنا الأساسي تاريخي، مش هنقدر ندّعي إن عندنا Real-Time Data حقيقية. بدل كده، هنكتب برنامج يقرأ السجلات واحدة واحدة، ويضيف لكل حدث وقت وصول محاكى.

مثلًا:

```
{
  "event_id": "evt-0001",
  "crop": "Wheat",
  "year": 2023,
  "supply_chain_stage": "Storage",
  "loss_percentage": 9.2,
  "event_time": "2023-06-01T00:00:00",
  "ingestion_time": "2026-10-13T14:00:01",
  "data_origin": "historical",
  "simulation": true
}
```

هنا فيه فرق بين `event_time` اللي بيمثل التاريخ المرتبط بالسجل الأصلي، و `ingestion_time` اللي بيمثل وقت وصوله للنظام أثناء المحاكاة.

هنستخدم Kafka لما نحتاج نفصل بين البرنامج اللي بيبعت الأحداث والبرامج اللي بتستقبلها وتعالجها. Kafka مصممة لنقل الأحداث والاحتفاظ بها وتوزيعها على المستهلكين.



Apache Kafka

ليه مش Kafka من أول خطوة؟

لأننا محتاجين الأول نتأكد إن شكل الحدث وقواعد التحقق واضحين. نقدر نختبر المنطق ببرنامج Python بسيط، وبعد نجاحه نربطه بـ Kafka من غير ما نخلط مشكلة صحة البيانات بمشكلة إعداد البنية التحتية.

Features جديدة:

- محاكاة وصول البيانات تدريجيًا.
- تسجيل وقت الوصول.
- تمييز البيانات التاريخية عن الصناعية.
- عدّ الأحداث المرسلة والمستقبلة.
- اختبار النظام على معدلات وصول مختلفة.

المرحلة السادسة: Stream Processing

القرار: Python Consumer في النسخة الأولى.

المستهلك هيستقبل الحدث ويعمل عليه خطوات واضحة:

1. يفكّ محتوى الرسالة.
2. يتحقق من وجود الحقول المطلوبة.
3. يتحقق من أنواع البيانات ونطاقات القيم.
4. يحدد إذا كان الحدث صالحًا أو محتاج مراجعة.
5. يمنع تكرار معالجة الحدث باستخدام معرّف مناسب.
6. يخزّن السجل أو يسجله في قائمة الأحداث المرفوضة.

هنضيف Spark Structured Streaming لو احتجنا معالجة مستمرة على حجم أكبر، أو عمليات تجميع زمنية وربط بين تدفقات متعددة. الأداة بتسمح بمعالجة البيانات المتدفقة تدريجيًا باستخدام واجهات DataFrame وSQL المألوفة.



Apache Spark

+1

ليه مش Spark من البداية؟

لأننا لسه بنبني إثباتًا عمليًا لصحة الـ Pipeline. تشغيل Spark بيحتاج إعدادًا وموارد إضافية، ومش شرط يضيف قيمة لو الحمل بسيط.

Features جديدة:

- معالجة الأحداث الواردة.
- اكتشاف الأحداث غير الصالحة.
- التعامل مع إعادة المحاولة.
- تسجيل الأخطاء.
- منع التكرارات قدر الإمكان.
- مراقبة عدد الأحداث اللي وصلت واتعالجت.

المرحلة السابعة: Analytics وData Transformation

القرار: SQL للتحليلات الأساسية، وPandas للتحليل الاستكشافي.

هنحسب مؤشرات زي:

- عدد السجلات لكل محصول.
- متوسط أو توزيع نسب الفقد حسب المرحلة.
- التغير في نسب الفقد بين السنوات.
- نسبة السجلات اللي فيها قيم مفقودة.
- اختلاف النتائج بين المصادر وطرق القياس.
- عدد الأحداث اللي اتعالجت وعدد الأحداث المرفوضة.

لكن مش هنحسب متوسطات عامة من غير ما نراجع معنى كل سجل. مثلًا، لو فيه مصدر بيقيس الفقد على مستوى منطقة ومصدر تاني على مستوى دولة، دمجهم في متوسط واحد ممكن ينتج مقارنة مضللة.

ليه مش dbt من البداية؟

لو عندنا عدد قليل من استعلامات SQL، نقدر ننظمها في ملفات واضحة. نضيف dbt لما التحويلات تكبر ونحتاج نماذج مترابطة واختبارات وتوثيقًا أفضل للـ SQL transformations.

Features جديدة:

- تقارير تحليلية قابلة للتحديث.
- مؤشرات حسب المحصول والسنة والمرحلة.
- مقارنة المصادر.
- توثيق قواعد حساب المؤشرات.
- فصل نتائج التحليل عن البيانات الخام.

المرحلة الثامنة: Machine Learning

القرار: Scikit-learn، بشرط إن البيانات تسمح.

قبل ما ندرّب أي نموذج، لازم نحدد بالضبط إحنا بنتنبأ بإيه.

فيه فرق بين:

- التنبؤ بنسبة الفقد لسجل جديد.
- تصنيف مستوى المخاطر إلى منخفض أو متوسط أو مرتفع.
- تفسير العوامل المرتبطة بالفقد.
- اكتشاف السجلات غير المعتادة.

مش كل مهمة من دول مناسبة لنفس البيانات أو لنفس المقاييس.

هنبدأ بنموذج بسيط Baseline، ونقارن أداءه بطريقة تقييم صحيحة. ولو البيانات مرتبطة بسنوات، هنراجع تقسيم التدريب والاختبار علشان نتجنب تسريب معلومات مستقبلية.

ليه مش XGBoost أو Deep Learning مباشرة؟

لأن تعقيد النموذج مش دليل على جودته. لازم الأول نتأكد من وجود Target موثوق، وعدد كافٍ من الأمثلة، وخصائص مناسبة، وطريقة تقييم تمثل الاستخدام المتوقع.

جديدة: نتيجة تنبؤ مصحوبة بمقياس أداء وحدود واضحة للتفسير، بدل عرض رقم تنبؤ من غير دليل على موثوقيته Feature.

ولو البيانات مش كافية، هنكمّل التحليلات الوصفية ونوثّق متطلبات البيانات اللازمة للنموذج بدل ما ندّعي دقة غير مثبتة.

المرحلة التاسعة: Dashboard

القرار: Power BI.

هنبني صفحات تعرض:

- ملخصًا لمؤشرات فقد الغذاء.
- نسب الفقد حسب المحصول والسنة والمرحلة.
- فلاتر حسب المصدر والمستوى الجغرافي لما يكون متاحًا.
- مؤشرات جودة البيانات.
- عدد الأحداث المستقبلة والمعالجة.
- وقت آخر تحديث.
- نتائج التنبؤ ومقاييس تقييم النموذج، لو اتنفذ.

ليه Power BI؟

لأن هدف المرحلة الأساسي هو استكشاف النتائج من خلال تقارير تفاعلية، وPower BI بيوفر رسومًا وفلاتر وربطًا بين البيانات والتقارير.



Microsoft Learn

ليه مش Streamlit؟

لو عايزين تطبيق Python مخصص يدمج الرسوم مع منطق المعالجة أو واجهة تشغيل للمحاكاة، Streamlit هيكون بديلًا مناسبًا. لكن مش محتاجين نبني Power BI وStreamlit مع بعض في أول نسخة إلا لو كل واحد فيهم بيقدم وظيفة مختلفة ومطلوبة.

Feature محاكاة أحداث تاريخية أو بيانات مبنية على التحديثات إذا كانت توضيح مع النتائج، يفهم المستخدم واجهة تساعد جديدة.

المرحلة العاشرة: Orchestration والاختبارات والنشر

القرار: pytest + Docker Compose، ثم Airflow عند الحاجة.

pytest: لاختبار قواعد التنظيف والتحويل ومعالجة الأحداث.

Docker Compose: لتشغيل قاعدة البيانات وKafka وباقي الخدمات المطلوبة في بيئة موحدة.

GitHub Actions: لتشغيل الاختبارات تلقائيًا عند رفع تغييرات جديدة.

Airflow: لإدارة المهام الدورية لما يكون عندنا Workflow متعدد الخطوات، مثل الاستيراد ثم التحقق ثم التحويل ثم التحديث.

Airflow مناسب لتنسيق المهام المجدولة ومتابعة تشغيلها وإعادة المهام الفاشلة.



Airflow 3.3.2 Documentation

ليه مش Airflow من أول يوم؟

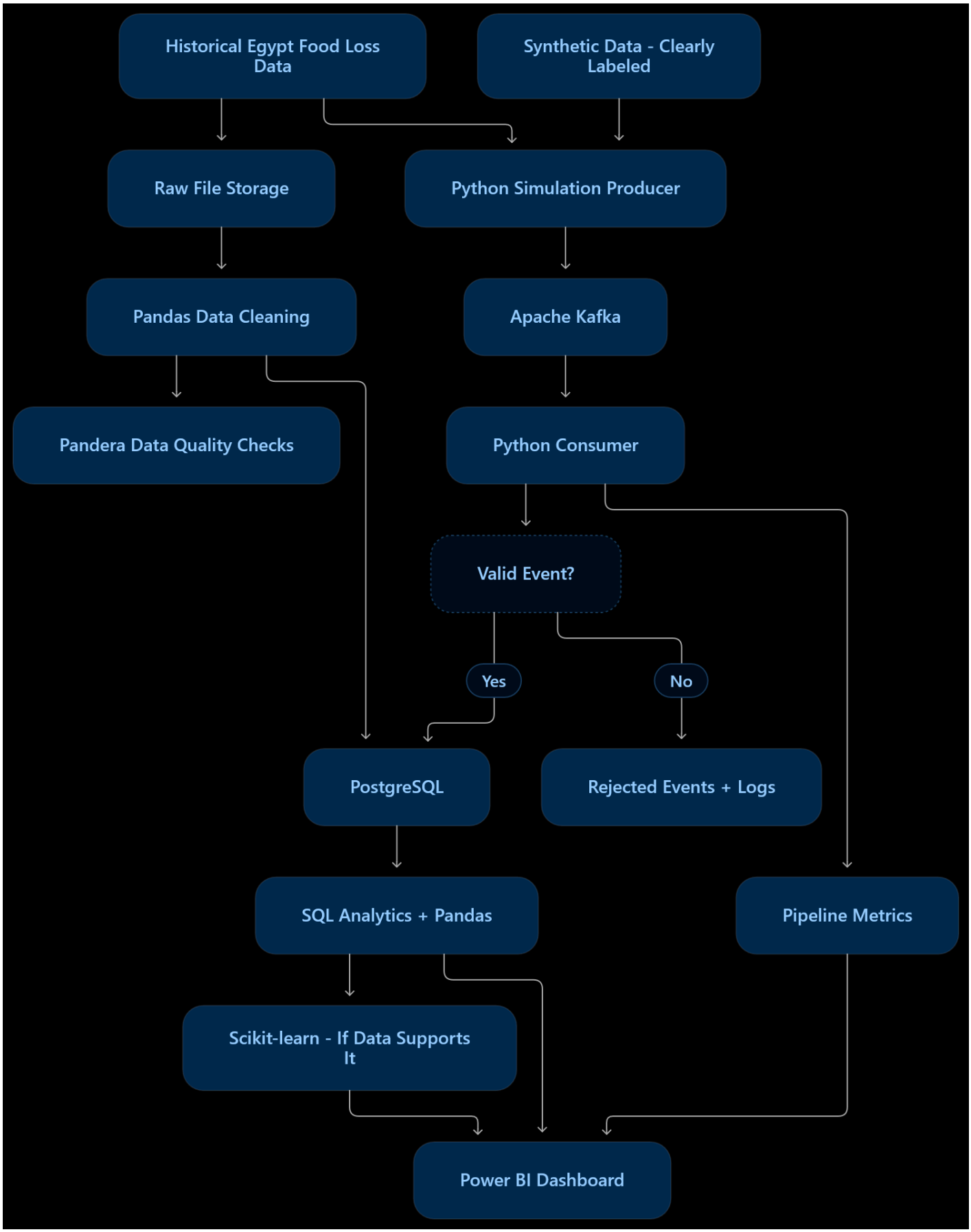
لأن تشغيل Scheduler وWorkers وواجهة إدارة لمهمة بسيطة واحدة ممكن يضيف عبئًا غير ضروري. هنبدأ باختبارات وتشغيل واضح، ونضيف Orchestration لما التعقيد الفعلي يبررها.

Features جديدة:

- اختبار آلي للتحويلات.
- تشغيل متكرر قابل للتوثيق.
- إعادة تشغيل المهام الفاشلة.
- تسجيل الأخطاء ومتابعة التنفيذ.
- بيئة تشغيل أسهل لإعادة التجربة.

3. الـ Architecture النهائية المقترحة

ده المسار اللي أنصح إننا نستهدفه، مع تنفيذ كل مكوّن تدريجيًا بدل ما نبني كل حاجة مرة واحدة.



في التصميم ده، البيانات التاريخية بتفضل محفوظة كمرجع، والبيانات الصناعية ليها علامة واضحة، وKafka مسؤولة عن نقل الأحداث، وPostgreSQL مسؤولة عن تخزين السجلات المنظمة، وPower BI مسؤولة عن عرض النتائج.

ممكن نضيف Spark أو Data Lake أو Airflow بعدين، لكن مش لازم نحطهم في المسار الأساسي قبل ما نثبت احتياجنا ليهم.

4. ترتيب التنفيذ الفعلي

الأولوية	هننفّذ إيه؟	إمتى نعتبره جاهز؟
1	فحص البيانات ومصادرها	نعرف الأعمدة ومعانيها ونطاق التغطية وجودة البيانات.
2	تنظيف البيانات وقواعد الجودة	نقدر نعيد تشغيل التنظيف ونحصل على نتائج موثقة.
3	تخزين الخام والمنظم	نحتفظ بالأصل ونقدر نستعلم عن البيانات المعالجة.
4	SQL Analytics	نطلع مؤشرات صحيحة ومفهومة ومراعية لاختلاف المصادر.
5	محاكاة الأحداث باستخدام Python	الأحداث بتوصل بالتدريج وبتمييز واضح للمصدر.
6	Kafka وConsumer	نقدر نبعت الأحداث ونستقبلها ونعالج المقبول ونرصد المرفوض.
7	Dashboard	المستخدم يقدر يتصفح المؤشرات والجودة وحالة الـ Pipeline.
8	Machine Learning	نضيف النموذج فقط لو البيانات مناسبة والتقييم قابل للدفاع عنه.
9	الاختبارات والتشغيل الآلي	نقدر نكرر التشغيل ونكتشف الأخطاء من غير مراجعة يدوية لكل خطوة.
10	التوسع	نضيف Spark أو Data Lake أو Airflow حسب قياسات الأداء والاحتياجات.

5. القرار النهائي للـ Tech Stack

ده اختياري الموصى به للمشروع، مش مجرد قائمة بكل الأدوات الممكنة:

الوظيفة	الاختيار

لغة البرمجة	Python
معالجة البيانات	Pandas
قواعد جودة البيانات	Pandera
الاحتفاظ بالبيانات الأصلية	ملفات CSV/JSON منظمة
قاعدة البيانات	PostgreSQL
محاكاة الأحداث	Python Producer
Event Streaming	Apache Kafka
معالجة الأحداث	Python Consumer
التحليلات	SQL + Pandas
Machine Learning	Scikit-learn، عند كفاية البيانات
Dashboard	Power BI
الاختبارات	pytest
الحاويات	Docker Compose
إدارة الكود	Git + GitHub

CI	GitHub Actions
Orchestration لاحقًا	Apache Airflow
معالجة موزعة لاحقًا	Spark Structured Streaming
تخزين كائنات على نطاق أكبر لاحقًا	Azure Data Lake Storage أو Amazon S3
المراقبة لاحقًا	Prometheus + Grafana